MWA Software

Pascal API 4 OpenSSL

Issue 2.0,
19 May 2026

MWA Software

EMail: info@ mccallumwhyman.com, http://www.mccallumwhyman.com

COPYRIGHT

Disclaimer

Although our best efforts have been made to ensure that the information contained within is up-to-date and accurate, no warranty whatsover is offered as to its correctness and readers are responsible for ensuring through testing or any other appropriate procedures that the information provided is correct and appropriate for the purpose for which it is used.

CONTENTS **Page**

1

Introduction

This document describes a Pascal Interface to the OpenSSL API. OpenSSL is itself an open source code library providing an implementation that includes the RFC 8446 Transport Layer Security (TLS) Protocol Version 1.3. (see https://www.openssl.org/).

The OpenSSL library is written in 'C' and the programmatic interface is defined in a series of 'C' header files. These have to be translated into Pascal units in order to declare the interface to a Pascal code library.

This translation is performed using MWA Software's *h2paspp* utility. This allows for automated translation of 'C' headers into Pascal units with only limited manual control needed i.e.

- Patches to ensure dependency order of constants and to resolve issues with conditional compilation
- directions to ensure
  - correct unit dependencies
  - Macro argument and return types where ambiguous
- Regular expressions that allow more efficient macro expansion to Pascal than which can reasonably be performed using automatic translation.

A (bash) script is used to manage the translation of different header versions. This script and for each header in each version

- Patches the 'C' header if a patch is provided for that header.
- Runs the 'C' preprocessor (gcc -E) to perform macro expansion
- Runs *h2paspp* to perform that actual translation.

The result is a set of header files translated from 'C' to Pascal:

- With comments in their original positions
- Conditional compilation directives passed through and translated to Pascal conditional compilation directives.
- And which support three different link models:
  - Compile time linkage to a shared (.so or .dll) library
  - Compile time linkage to a static library (libssl.a and libcrypto.a)
  - Dynamic Library Load and the run time resolution of each API call used.

The “headers” folder in the source code distrbution contains generated headers for each OpenSSL release translated. Additionally, the “headersWithLegacySupport” folder contains the OpenSSL 3.0 headers with additional support for backwards compatibility for OpenSSL 1.0.2 and 1.1.0.

Note. Compatibility is limited to core functions only and specifically those needed to support the Indy Library.

The headers are derived from the OpenSSL source code and hence are distributed using the same Apache License v2.0 as OpenSSL itself.

Note. The API is large (over 191K lines for the OpenSSL 4.0.0 Pascal Headers) and while the core functionality has been tested using the IndySecOpenSSL package,it is not currently feasible to extensively test all translated headers other than by a “Clean Compile”. Users must obtain their own confidence in their use of these headers through their own testing. The Pascal OpenSSL headers are distributed in the hope that they may be useful but with no warrantee whatsoever.

2

Using the OpenSSL API

The header files for whichever OpenSSL release is appropriate for you should be copied into your project/package. It is recommended that they are all placed in a dedicated subdirectory and, the project options are set such that the unit and include file search paths include this subdirectory.

Note. In order to avoid linking in OpenSSL header units unnecessarily, ideally these units should not be explicitly added to your project or package. Only the search path should be specified.

In order to use an API call it is sufficient to include the appropriate header file unit name in your unit's uses clause and call the API call as a normal function or procedure. This is regardless of whether static or dynamic linking is used. Each API call has the same name as the original OpenSSL 'C' API call and the header file unit names follow the OpenSSL header naming conventions. The OpenSSL Documentation should be consulted for the semantics of each API call.

Note. See Appendix A for handling of case insensitive name clashes.

With dynamic linking, there is no need to explicitly load the library prior to use as it is implicitly loaded and initialised when the first API call is called. With static linking, the library must be explicitly initialised prior to use (see 2.3.1).

2.1 Selection of the OpenSSL Release

If in doubt, you should use the most recent OpenSSL 3.0.x headers (3.0.20 at the time of writing). These include all core functionality and are typically compatible with later OpenSSL releases (although this is not guaranteed and runtime testing is advised).

Note. Currently, it appears that OpenSSL 4.0.0 libraries are backwards compatible with OpenSSL 3.0.20 headers - at least for common API calls.

If you need specific functionality only available in later releases of OpenSSL then, in most cases, you should use the headers from the release from which this functionality was added.

2.2 Compile Time Code Selection

The following header file options may be selected at compile time.

2.2.1 Link Strategy Selection

If in doubt, dynamic linking is recommended.

If static linking is required then your program (or package) must be compiled with the defined symbol:

- OPENSSL_USE_STATIC_LIBRARY, or
- OPENSSL_USE_SHARED_LIBRARY.

The former selects linking to static OpenSSL libraries (e.g. libssl.a and libcrypto.a), while the latter selects linking to shared OpenSSL libraries (.so/.dll).

The main difference between the two is that static OpenSSL Libraries are linked into the program executable by the linker and are hence distributed as part of the program executable. In the later case, the shared library is loaded and linked to your program executable at program load time. The former has the advantage that your can guarantee the version of OpenSSL that your application uses. The later minimises the size of the executable whilst adds the risk that an unexpected version of the OpenSSL libraries are used.

When linking to static libraries, the library version should be the same as the headers. When linking to shared libraries, the OpenSSL library (dll or so) should be the same or a later version (assuming backwards compatibility).

Dynamic linking is the default (neither symbol is defined). In this case, the dynamic library loader (in the OpenSSLAPI unit) searches for the shared OpenSSL libraries and attempts to load the most recent compatible with the headers. It is generally the most flexible option.

Otherwise, this strategy's advantages and disadvantages are the same as for linking to shared libraries at program load time, except that program load time should be minimised and your program should not fail to load if the OpenSSL shared library version on the target system does not include an API call that is included in the header files but is not used by your application.

Note: The OpenSSLAPI unit is always included by default if an OpenSSL Header unit is included in your project.

2.2.2 Legacy Support

Primarily in support of Indy, header files proving legacy support include compatibility functions to enable the use of OpenSSL from release 1.0.2 or later (with dynamic library load). These compatibility functions replace some OpenSSL 3.0 API calls when not found to be present on load, with functions that call equivalent functions in earlier versions of OpenSSL and which emulate the missing 3.0 API call.

If your application only uses OpenSSL 3.0.0 and 3.0.0 or later libraries then the compatibility functions can be ignored at compile time by compiling with the defined symbol:

OPENSSL_NO_LEGACY_SUPPORT

or by simply using the standard release 3.0.20 headers. This should avoid unnecessary program bloat.

2.2.3 Memory Management

OpenSSL's internal memory management functions can be replaced by the Pascal Memory Manager by compiling with the defined symbol:

OPENSSL_SET_MEMORY_FUNCS

This may be useful for debugging e.g. reporting memory leaks on completion.

2.2.4 Minimum Library Version Checking

By default, the OpenSSL API dynamic library loader checks the library version and raises an exception if the major and minor version numbers are less that of the header files used (or 1.0.2 for legacy support). This can be turned off by compiling with the defined symbol

OPENSSL_NO_MIN_VERSION

Note. If this option is not turned off then the effect is to force the OpenSSL shared library minimum version number to be the same as the header files or later.

2.3 Run Time Interface

The OpenSSLAPI unit provides two Pascal COM interfaces which provide information and option selection for the OpenSSL Library.

2.3.1 The IOpenSSL Interface

This interface is available for all link strategies and the interface is accessed using the function:

```
function GetIOpenSSL: IOpenSSL;
```

The interface is declared as:

TOpenSSL_LinkModel = (lmDynamic, lmShared, lmStatic);

```
IOpenSSL = interface
['{aed66223-1700-4199-b1c5-8222648e8cd5}']
  function GetOpenSSLPath: string;
  function GetOpenSSLVersionStr: string;
  function GetOpenSSLVersion: TopenSSL_C_ULONG;
  function GetLinkModel: TOpenSSL_LinkModel;
  function Init: boolean;
end;
```

`function GetOpenSSLPath: string`	Static Linking: Returns the OpenSSL installation path as compiled into the OpenSSL library. Dynamic Loading: Returns • The same as above when the library is loaded

	• the specified OpenSSL installation path, otherwise. *Note. May be empty.*
`function GetOpenSSLVersionStr: string`	Returns the OpenSSL library version string. e.g “OpenSSL 3.2.0 23 Nov 2023”
`function GetOpenSSLVersion: TOpenSSL_C_ULONG`	Returns the OpenSSL library version as an integer as defined by the OpenSSL documentation.
`function GetLinkModel: TOpenSSL_LinkModel;`	Returns the Link Model used. Values: lmDynamic, lmShared, lmStatic
`function Init: boolean`	Called to initialise the OpenSSL library prior to use. This must be called when using static linking. It is optional for dynamic linking (implicitly called on library load). Returns true on success.

2.3.1.1 Windows OpenSSL Library Name

By default, the Windows DLL names for the OpenSSL libraries default to:

- libcrypto-3-x64.dll and libssl-3-x64.dll (64-bit), or
- libcrypto-3.dll and libssl-3.dll (32-bit)

If the symbol USE_OPENSSL_VERSION_4 is defined then the Windows DLL names for the OpenSSL libraries default to:

- libcrypto-4-x64.dll and libssl-4-x64.dll (64,bit) or
- libcrypto-4.dll and libssl-4.dll (32 bit)

When OpenSSL 4.x.x headers are used then USE_OPENSSL_VERSION_4 should be automatically defined.

2.3.2 The IOpenSSLDLL Interface

This interface is only available when the API is configured at compile time for dynamic library loading. The interface is accessed using the function:

```
function GetIOpenSSLDDL: IOpenSSLDLL;
```

This function returns “nil” if the interface is not available. This may be used as a runtime test to determine if dynamic library loading if been configured.

Note The OpenSSLAPI unit only declares the constant

```
OpenSSL_Using_Dynamic_Library_Load = true;
```

when configured at compile time for dynamic library loading. The constant is not declared otherwise. This feature may also be used as a compile time test for the dynamic library loading strategy e.g.

```
{$if declared(OpenSSL_Using_Dynamic_Library_Load)}
…
{$ifend}
```

Similarly, the constant

```
OpenSSL_Using_Shared_Library = true;
```

is only declared when using the share library link model.

The interface includes the IOpenSSL interface and is declared as:

```
  IOpenSSLDLL = interface(IOpenSSL)
    procedure SetOpenSSLPath(const Value: string);
    function GetSSLLibVersions: string;
    procedure SetSSLLibVersions(AValue: string);
    function GetSSLBaseLibName: string;
    procedure SetSSLBaseLibName(AValue: string);
    function GetCryptoBaseLibName: string;
    procedure SetCryptoBaseLibName(AValue: string);
    function GetAllowLegacyLibsFallback: boolean;
    procedure SetAllowLegacyLibsFallback(AValue: boolean);
    function GetLibCryptoHandle: TLibHandle;
    function GetLibSSLHandle: TLibHandle;
    function GetLibCryptoFilePath: string;
    function GetLibSSLFilePath: string;
    function Load: Boolean;
    procedure Unload;
    function IsLoaded: boolean;
    property SSLLibVersions: string read GetSSLLibVersions write SetSSLLibVersions;
    property SSLBaseLibame: string read GetSSLBaseLibName write SetSSLBaseLibName;
    property CryptoBaseLibName: string read GetCryptoBaseLibName
                                      write SetCryptoBaseLibName;
    property AllowLegacyLibsFallback: boolean read GetAllowLegacyLibsFallback
                                     write SetAllowLegacyLibsFallback;end;
```

`procedure SetOpenSSLPath(const Value: string);`	This sets the OpenSSLPath used to locate the OpenSSL library modules (e.g. libcrypto.so). It needs to be set when OpenSSL has not been installed in a default location and/or multiple versions of OpenSSL have been installed on the same system. This may be empty (default), contain a single path, or a colon (Unix) or semi-colon (Windows) separated list of paths to try in order.

`function GetSSLLibVersions: string;`
 	
`procedure SetSSLLibVersions(AValue: string);`
	This is a colon separated (Unixes) or semi-colon separated (Windows) ordered list of OpenSSL version numbers that can be used as suffices for the OpenSSL library modules (e.g. for libcrypto-3-x64.dll, the suffix is '-3-x64').
	
	When searching for the OpenSSL library, the loader searches the default (or specified) OpenSSLPath for OpenSSL libraries using these suffices in turn. For dynamic loading:
	
	Unix: defaults to
	
	'.4:.3:.1.1:.1.0.2:.1.0.0:.0.9.9:.0.9.8:.0.9.7:.0.9.6'
	
	Note includes legacy versions.
	
	Windows defaults to
	
	'-4-x64;-3-x64;-1-x64;' (64 bit)
	
	'-4;-3;-1;' (32 bit)
	
	Changing the libversions will automatically unload the ssl and crypto libraries if currently loaded.

`function GetSSLBaseLibName: string;`
 	
`procedure SetSSLBaseLibName(AValue: string);`
	These are used to respectively get and set the basename for the SSL dynamic library. Defaults to 'libssl'.
	
	Changing the base name will automatically unload the ssl and crypto libraries if currently loaded.

`function GetCryptoBaseLibName: string;`
 	
`procedure SetCryptoBaseLibName(AValue: string);`
	These are used to respectively get and set the basename for the crypto dynamic library. Defaults to 'libcrypto'.
	
	Changing the base name will automatically unload the ssl and crypto libraries if currently loaded.

`function GetAllowLegacyLibsFallback: boolean;`
 	
`procedure SetAllowLegacyLibsFallback(AValue: boolean);`
	These are used to respectively get and set the AllowLegacyLibsFallback flag. This is only used when running under Windows.
	
	If set and no OpenSSL libraries have been found when searching for them using the Base Library names and libversions, then the loader will attempt to load the OpenSSL libraries using the legacy library names 'libeay32' and 'ssleay32'.
	
	Defaults to 'false' unless the legacy support headers are used when it defaults to 'true'.
	
	Changing this flag will automatically unload the ssl and crypto libraries if currently loaded.

`function GetLibCryptoHandle: TLibHandle`	After a successful library load, this returns the value of the internal handle to libcrypto (internal use only recommended). *Note. Library load attempted if the library has not been successfully loaded prior to the call.*
`function GetLibSSLHandle: TLibHandle;`	After a successful library load, this returns the value of the internal handle to libssl (internal use only recommended). *Note. Library load attempted if the library has not been successfully loaded prior to the call.*
`function GetLibCryptoFilePath: string`	After a successful library load, this returns the path to the loaded libcrypto library. (note: may be empty if default library loaded).
`function GetLibSSLFilePath: string`	After a successful library load, this returns the path to the loaded libssl library. (note: may be empty if default library loaded).
`function Load: Boolean;`	Explicitly loads the library if not already loaded and returns “true” on successful load. *Note. This is normally invoked automatically when the first API call is called but may be called explicitly to force library load (e.g. on program startup).*
`procedure Unload;`	Explicitly unloads the library if current loaded.

`function IsLoaded: boolean`	Returns true if the OpenSSL library has been successfully loaded.

2.3.3 Detecting the Header Version

Separate to determining the OpenSSL release number for the loaded library, it is also possible to determine the OpenSSL release from which the headers where generated. This is enabled by including the “openssl_opensslv.inc” header in the interface section of the OpenSSLAPI unit.

For example, the constant OPENSSL_VERSION_STR will then return the OpenSSL header version (e.g. '3.0.20'). Other similar constants are also available.

2.4 Windows Trusted Certificate Store Support

So that OpenSSL can verify certificates, access is required to a list of trusted root certificates which can be loaded into its internal certificate store prior to certificate validation.

When OpenSSL is built, a default location for this list is configured into the software and any list of certificates found there is, at run time, loaded when the user invokes the API function:

SSL_CTX_set_default_verify_paths

Under Linux, a suitable list of trusted root certificates is distributed with the Linux distro and located in a well known location (e.g . on Debian derived distributions this is /etc/ssl/certs). It is sufficient for the API user to invoke the above function call in order to enable certificate verification.

Under Windows, a list of trusted root certificates is similarly distributed with the Operating System but rather than locate this in some well known directory, the list is secured in a small secure database.

It is possible for a list of trusted certificates to be distributed with an application or OpenSSL itself and placed in the configured well known location (and then loaded using the SSL_CTX_set_default_verify_paths API call). However, this is rarely done and, it may be argued, that the Windows secure database should, in most cases, be the preferred up-to-date source of trusted root certificates, given that it is maintained by Windows Update.

In order to support use of the Windows trusted root certificate store, the PascalAPI4OpenSSL comes with an additional unit “openssl_win509.pas”. This unit contains the functionality needed to load OpenSSL's internal certificate store from the Windows trusted root certificate store.

When compiled under Windows, this unit provides the following additional API calls:

```
function HasWindowsCertStore: boolean;
function LoadWindowsCertStore(ctx:PSSL_CTX): integer;
```

The HasWindowsCertStore returns true if the Windows trusted root certificate store is available and false otherwise.

The LoadWindowsCertStore function loads OpenSSL's internal certificate store from the Windows trusted root certificate store and returns the number of certificates loaded.

2.4.1 Using the openssl_winx509 unit

The LoadWindowsCertStore function should be called soon after a new SSL context has been created, and for each new SSL context e.g.

```
uses openssl_types, openssl_x509;

var ctx: PSSL_CTX;

…
  ctx := SSL_CTX_new(<SSL Method>);

…
{$IF declared(HasWindowsCertStore)}
  if HasWindowsCertStore then
    LoadWindowsCertStore(ctx);
{$ELSE}
  SSL_CTX_set_default_verify_paths(ctx);
{$IFEND}
```

The openssl_x509 is safe to include for all target platforms. It is an empty unit on non-Windows platforms.

The conditional code ensures that the openssl_x509 functions are only called when available. Hence the above should be valid for all platforms. Typically, on non-Windows platforms, SSL_CTX_set_default_verify_paths should be called instead in order to load OpenSSL's internal certificate store from the default location.

The action taken if HasWindowsCertStore returns false is application dependent. For example, you may decide to raise an exception or to try the default location.

2.4.2 openssl_winx509 unit Link strategies

The Windows trusted certificate store is access using the 'crypt32.dll' link library.

This is loaded dynamically at OpenSSL library load time if the Dynamic Library Load link strategy is in use. Otherwise, linking is performed at program load (i.e. Compile time linkage to a shared library).

3

Library Linking Strategies

In the Indy component library, the default linking strategy for OpenSSL was to load the OpenSSL libraries dynamically at runtime and to resolve each API call entry point using the 'GetProcAddress' function call and immediately afterwards. The returned address was then assigned to a function variable with an appropriate type.

The downsides of this “legacy” approach are that:

- It may take a noticeable time to load all API call addresses including many that are never used.
- An error is typically declared if an API call fails to be loaded even when the call is never used by the user program.
- There is no overall strategy for managing multiple versions of the OpenSSL library with different sets of API calls.
- Static linking is possible but only by replacing the dynamic loading units with a static linkage unit and using conditional compilation in user code to manage the differences.

These headers are intended to avoid these problems, and use conditional compilation to support three separate linking strategies in the same unit and transparently to the user program. Both static and dynamic linking are supported.

Note. With static linking, the OpenSSL library must be explicitly initialised before use (see 2.3.1). With dynamically loading the library is automatically loaded and initialised on the first API call.

3.1 Static Linking

Two flavours of static linking are supported:

- Linking to a static library (typically with a “*.a”* suffix).

- Linking to a shared library (*.so* or *.dll*) with call API resolution by the Operating System's program loader at program load time.

With static linking, a specific OpenSSL version has to be supported, as determined by the OpenSSL release used to generate the selected headers. Only the API calls present in this version can be declared as external function/procedure calls.

Note. This restriction is a consequence of the linking/loading of the program failing if not all declared API calls are present in the external library.

3.1.1 Linking to a static library

Internally and in order to achieve this:

- each API call is formatted as an external declaration with an empty library name.
- FPC: a {$LINKLIB …} compiler directive is provided for each external library to which the compiled unit is to be linked (e.g. {$LINKLIB ssl.a} {$LINKLIB crypto.a}).
- Delphi: a {$LINK …} compiler directive will need to be provided for each external object file to which the compiled unit is to be linked. The object files should follow the expected naming convention (i.e. <unitname>.obj).
    - For 32-bit Windows, the named file must be an Intel relocatable object file (OMF86), or 32-bit COFF format object file.
    - For 64-bit Windows, ELF objects generated by bcc64, and COFF objects by Visual C++, can be recognized.

3.1.2 Linking to a shared library

Internally, and in order to achieve this:

- each API call must be formatted as an external declaration with the name of the shared library in which the API call is located.

3.1.3 Implementation

h2paspp generates a conditional compilation section in each output unit's interface section for static linking providing:

- Each API call as an external declaration with a named constant used for the library name.

Depending on the static linking model this constant is either empty (static linking) or is set to the library name (shared library linking).

The OpenSSLAPI unit (include file) defines common types used by the API calls. It also includes:

- FPC only: a {$LINKLIB …} directive for standard OpenSSL libraries (e.g. those generated by gcc).
- The definitions of the named constants used for library names. These constants are empty strings when linking to a static library, while their value is set to the library names when linking to a shared library.

The user of the headers need only set the appropriate symbol for the required link model. The above is for information only.

Note. Unit file template includes a conditional section for Delphi and static linking only that includes a $LINK directive for the corresponding object file.

3.2 Dynamic Linking

h2paspp generates a conditional compilation section in each output unit's interface section for dynamic linking and as the alternative to static linking.

The dynamic linking model uses “Just in Time Linking (JIT)”: At compile time, each API call is set to a generated loader function specific to the API call. The first time a user calls the API call, the loader function resolves the API call from the link library and replaces the customised loader function address in the API Call function variable with the loaded function address. The API call is then invoked.

If the API call is not present in the library, then an EOpenSSLAPIFunctionNotPresent exception is raised.

Note that when legacy support is provided and when the API call is not present in the link library then a legacy support function is called instead, if it has been declared. Legacy support functions are declared in a separate include file only included when legacy support is enabled.

Note. In the h2paspp configuration file, it is also possible to label some API class as “Allow Nil”. In this case, the var declarations is initialised to 'nil'. The purpose of this is to allow the presence of the API call to be determined by checking for a non-nil value rather than having to catch an exception See Appendix B).

3.2.1 Implementation

In conditional compilation section for dynamic loading in each output unit's interface section:

- *Var* declarations are generated for each API call and each variable is initialised to the address of the API Call's generated loader function (or nil for “allow nil” functions - see above).

In the implementation section:

- the loader function body for each API call is generated.
- A “load” function is declared which attempts to use “GetProcAddress” to load each API call in turn for each “allow nil” API function. This ensures that the correct status of the API call is known as soon as the library has been loaded.
- An “unload” function is declared which resets each API function var to its initial value.
- Load Function Registration:

  At unit initialisation time, both the Load and Unload functions are registered with the library loader (in the OpenSSLAPI unit) by a call to the registration function “Register_SSLLoader”.

3.3 The OpenSSLAPI Unit

The unit “OpenSSLAPI.pas” includes the library loader. The loader is also subject to conditional compilation and only included when dynamic linking is used.

The library loader loads both OpenSSL libraries (libssl and libcrypto) and then calls each registered load function in turn.

The library loader does not have to be explicitly called for the dynamic linking model but should be called before any API function vars are tested for a non-nil value if this feature is used (see 2.3.2)

4

Notes on the OpenSSLh2pas.sh Script

This shell script may be found in the “gen” folder and is used to generate each set of headers for a given OpenSSL release. The script is driven by a configuration file also in the “gen” folder with a separate configuration file for each release.

The script is called with options:

```
./OpenSSLh2pas.sh [-h] [-o <output directory>] [-d] [-C] Configureation filename>]
<source dir/filename>
```

The options are:

-o	Header output directory (default set in configuration file)
-h	Prints out option summary
-d	Debug mode
-C	The name of the configuration file (defaults to “openssl.cfg” in the current directory).
<source dir/filename>	Source filename (.h extension assumed) or source directory for multiple files

4.1 Configuration File

This file is included in the script at run time and is used to set variables that control the operation of the script. For example, the openssl3.0.cfg configuration file is used to generate the 3.0.x headers and has the content:

```
DEFAULTMAJORMINOR="3.0"
DEFAULTVERNUM="3.0.20"
DEFAULTPATCHLEVEL=
LIBDIR="${DEFAULTVERNUM}${DEFAULTPATCHLEVEL}"
DEFAULTOPENSSL="$HOME/SoftwareDev/external/openssl/openssl-${LIBDIR}"
DEFAULTHDRDIR="${DEFAULTOPENSSL}/include/openssl"
DEFAULTOUTPUTROOT=../headers
VERHDR=opensslv.h
H2PAS=h2paspp
ROOTDIR=`dirname $0`
H2PASCONFIG="${ROOTDIR}/data/${DEFAULTMAJORMINOR}/h2pasoptions.cfg"
IGNOREINCLUDEFILES=spthread
IGNOREHEADERS=asn1_mac.h
OTHERFILES="${ROOTDIR}/data/OtherFiles"
CPATCHDIR="${ROOTDIR}/data/${DEFAULTMAJORMINOR}/patches/${LIBDIR}"
PPATCHDIR="${CPATCHDIR}"
CONFIGDIR="${ROOTDIR}/data"
```

- DEFAULTMAJORMINOR and DEFAULTVERNUM determine the release and are different for each release.
- DEFAULTPATCHLEVEL is used when the OpenSSL library has a patch level. For example he latest OpenSSL 1.0.2 has a patch level of 'u'.
- DEFAULTOPENSSL gives the source directory for the OpenSSL release. This is dependent on the directory layout on the system used to generate the headers and will to be changed to whatever location is used on your system.

The remaining options are rarely changed.

Appendix A - List of Modified Constant Names

Unlike 'C', Pascal is a case-insensitive programming language. In consequence, there are cases where two identifiers in the 'C' headers differ only in case and which while these are unique in 'C', these are not unique identifiers in Pascal. In order to handle this case, *h2paspp* adds an extra underscore to one of the identifiers - typically a constant where a type or function name and a constant name differ only in case. The following in a list of unit and identifiers to which an underscore has been added in the 4.0.0 headers.

```
openssl_aes.pas:  AES_ENCRYPT_ = 1;
openssl_aes.pas:  AES_DECRYPT_ = 0;
openssl_bio.pas:  BIO_CTRL_PENDING_ = 10;
openssl_bio.pas:  BIO_CTRL_WPENDING_ = 13;
openssl_blowfish.pas:  BF_ENCRYPT_ = 1;
openssl_blowfish.pas:  BF_DECRYPT_ = 0;
openssl_camellia.pas:  CAMELLIA_ENCRYPT_ = 1;
openssl_camellia.pas:  CAMELLIA_DECRYPT_ = 0;
openssl_cast.pas:  CAST_ENCRYPT_ = 1;
openssl_cast.pas:  CAST_DECRYPT_ = 0;
openssl_cms.pas:  CMS_STREAM_ = $1000;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_GETTABLE_PARAMS_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_GET_PARAMS_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_THREAD_START_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_GET_LIBCTX_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_NEW_ERROR_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_SET_ERROR_DEBUG_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_VSET_ERROR_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_SET_ERROR_MARK_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_CLEAR_LAST_ERROR_MARK_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_POP_ERROR_TO_MARK_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_OBJ_ADD_SIGID_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_OBJ_CREATE_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_MALLOC_ = 20;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_ZALLOC_ = 21;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_FREE_ = 22;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_CLEAR_FREE_ = 23;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_REALLOC_ = 24;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_CLEAR_REALLOC_ = 25;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_SECURE_MALLOC_ = 26;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_SECURE_ZALLOC_ = 27;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_SECURE_FREE_ = 28;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_SECURE_CLEAR_FREE_ = 29;
openssl_core_dispatch.pas:  OSSL_FUNC_CRYPTO_SECURE_ALLOCATED_ = 30;
openssl_core_dispatch.pas:  OSSL_FUNC_OPENSSL_CLEANSE_ = 31;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_NEW_FILE_ = 40;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_NEW_MEMBUF_ = 41;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_READ_EX_ = 42;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_WRITE_EX_ = 43;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_UP_REF_ = 44;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_FREE_ = 45;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_VPRINTF_ = 46;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_VSNPRINTF_ = 47;
```

```
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_PUTS_ = 48;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_GETS_ = 49;
openssl_core_dispatch.pas:  OSSL_FUNC_BIO_CTRL_ = 50;
openssl_core_dispatch.pas:  OSSL_FUNC_CLEANUP_USER_ENTROPY_ = 96;
openssl_core_dispatch.pas:  OSSL_FUNC_CLEANUP_USER_NONCE_ = 97;
openssl_core_dispatch.pas:  OSSL_FUNC_GET_USER_ENTROPY_ = 98;
openssl_core_dispatch.pas:  OSSL_FUNC_GET_USER_NONCE_ = 99;
openssl_core_dispatch.pas:  OSSL_FUNC_INDICATOR_CB_ = 95;
openssl_core_dispatch.pas:  OSSL_FUNC_SELF_TEST_CB_ = 100;
openssl_core_dispatch.pas:  OSSL_FUNC_GET_ENTROPY_ = 101;
openssl_core_dispatch.pas:  OSSL_FUNC_CLEANUP_ENTROPY_ = 102;
openssl_core_dispatch.pas:  OSSL_FUNC_GET_NONCE_ = 103;
openssl_core_dispatch.pas:  OSSL_FUNC_CLEANUP_NONCE_ = 104;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_REGISTER_CHILD_CB_ = 105;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_DEREGISTER_CHILD_CB_ = 106;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_NAME_ = 107;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_GET0_PROVIDER_CTX_ = 108;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_GET0_DISPATCH_ = 109;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_UP_REF_ = 110;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_FREE_ = 111;
openssl_core_dispatch.pas:  OSSL_FUNC_CORE_COUNT_TO_MARK_ = 120;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_TEARDOWN_ = 1024;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_GETTABLE_PARAMS_ = 1025;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_GET_PARAMS_ = 1026;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_QUERY_OPERATION_ = 1027;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_UNQUERY_OPERATION_ = 1028;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_GET_REASON_STRINGS_ = 1029;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_GET_CAPABILITIES_ = 1030;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_SELF_TEST_ = 1031;
openssl_core_dispatch.pas:  OSSL_FUNC_PROVIDER_RANDOM_BYTES_ = 1032;
openssl_core_dispatch.pas:  OSSL_FUNC_SSL_QUIC_TLS_CRYPTO_SEND_ = 2001;
openssl_core_dispatch.pas:  OSSL_FUNC_SSL_QUIC_TLS_CRYPTO_RECV_RCD_ = 2002;
openssl_core_dispatch.pas:  OSSL_FUNC_SSL_QUIC_TLS_CRYPTO_RELEASE_RCD_ = 2003;
openssl_core_dispatch.pas:  OSSL_FUNC_SSL_QUIC_TLS_YIELD_SECRET_ = 2004;
openssl_core_dispatch.pas:  OSSL_FUNC_SSL_QUIC_TLS_GOT_TRANSPORT_PARAMS_ = 2005;
openssl_core_dispatch.pas:  OSSL_FUNC_SSL_QUIC_TLS_ALERT_ = 2006;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_INIT_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_UPDATE_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_FINAL_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_DIGEST_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_FREECTX_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_DUPCTX_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_GET_PARAMS_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_SET_CTX_PARAMS_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_GET_CTX_PARAMS_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_GETTABLE_PARAMS_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_SETTABLE_CTX_PARAMS_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_GETTABLE_CTX_PARAMS_ = 13;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_SQUEEZE_ = 14;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_COPYCTX_ = 15;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_SERIALIZE_ = 16;
openssl_core_dispatch.pas:  OSSL_FUNC_DIGEST_DESERIALIZE_ = 17;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_ENCRYPT_INIT_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_DECRYPT_INIT_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_UPDATE_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_FINAL_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_CIPHER_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_FREECTX_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_DUPCTX_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_GET_PARAMS_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_GET_CTX_PARAMS_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_SET_CTX_PARAMS_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_GETTABLE_PARAMS_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_GETTABLE_CTX_PARAMS_ = 13;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_SETTABLE_CTX_PARAMS_ = 14;
```

```
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_PIPELINE_ENCRYPT_INIT_ = 15;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_PIPELINE_DECRYPT_INIT_ = 16;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_PIPELINE_UPDATE_ = 17;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_PIPELINE_FINAL_ = 18;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_ENCRYPT_SKEY_INIT_ = 19;
openssl_core_dispatch.pas:  OSSL_FUNC_CIPHER_DECRYPT_SKEY_INIT_ = 20;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_DUPCTX_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_FREECTX_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_INIT_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_UPDATE_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_FINAL_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_GET_PARAMS_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_GET_CTX_PARAMS_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_SET_CTX_PARAMS_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_GETTABLE_PARAMS_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_GETTABLE_CTX_PARAMS_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_SETTABLE_CTX_PARAMS_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_MAC_INIT_SKEY_ = 13;
openssl_core_dispatch.pas:  OSSL_FUNC_SKEYMGMT_FREE_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_SKEYMGMT_IMPORT_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_SKEYMGMT_EXPORT_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_SKEYMGMT_GENERATE_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_SKEYMGMT_GET_KEY_ID_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_SKEYMGMT_IMP_SETTABLE_PARAMS_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_SKEYMGMT_GEN_SETTABLE_PARAMS_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_DUPCTX_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_FREECTX_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_RESET_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_DERIVE_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_GETTABLE_PARAMS_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_GETTABLE_CTX_PARAMS_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_SETTABLE_CTX_PARAMS_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_GET_PARAMS_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_GET_CTX_PARAMS_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_SET_CTX_PARAMS_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_SET_SKEY_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_KDF_DERIVE_SKEY_ = 13;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_FREECTX_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_INSTANTIATE_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_UNINSTANTIATE_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_GENERATE_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_RESEED_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_NONCE_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_ENABLE_LOCKING_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_LOCK_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_UNLOCK_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_GETTABLE_PARAMS_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_GETTABLE_CTX_PARAMS_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_SETTABLE_CTX_PARAMS_ = 13;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_GET_PARAMS_ = 14;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_GET_CTX_PARAMS_ = 15;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_SET_CTX_PARAMS_ = 16;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_VERIFY_ZEROIZATION_ = 17;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_GET_SEED_ = 18;
openssl_core_dispatch.pas:  OSSL_FUNC_RAND_CLEAR_SEED_ = 19;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_NEW_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_NEW_EX_ = 17;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GEN_INIT_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GEN_SET_TEMPLATE_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GEN_SET_PARAMS_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GEN_SETTABLE_PARAMS_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GEN_ = 6;
```

```
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GEN_CLEANUP_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GEN_GET_PARAMS_ = 15;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GEN_GETTABLE_PARAMS_ = 16;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_LOAD_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_FREE_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GET_PARAMS_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_GETTABLE_PARAMS_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_SET_PARAMS_ = 13;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_SETTABLE_PARAMS_ = 14;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_QUERY_OPERATION_NAME_ = 20;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_HAS_ = 21;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_VALIDATE_ = 22;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_MATCH_ = 23;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_IMPORT_ = 40;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_IMPORT_TYPES_ = 41;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_EXPORT_ = 42;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_EXPORT_TYPES_ = 43;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_DUP_ = 44;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_IMPORT_TYPES_EX_ = 45;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYMGMT_EXPORT_TYPES_EX_ = 46;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_INIT_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_DERIVE_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_SET_PEER_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_FREECTX_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_DUPCTX_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_SET_CTX_PARAMS_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_SETTABLE_CTX_PARAMS_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_GET_CTX_PARAMS_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_GETTABLE_CTX_PARAMS_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_KEYEXCH_DERIVE_SKEY_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SIGN_INIT_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SIGN_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_VERIFY_INIT_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_VERIFY_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_VERIFY_RECOVER_INIT_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_VERIFY_RECOVER_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DIGEST_SIGN_INIT_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DIGEST_SIGN_UPDATE_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DIGEST_SIGN_FINAL_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DIGEST_SIGN_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DIGEST_VERIFY_INIT_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DIGEST_VERIFY_UPDATE_ = 13;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DIGEST_VERIFY_FINAL_ = 14;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DIGEST_VERIFY_ = 15;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_FREECTX_ = 16;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_DUPCTX_ = 17;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_GET_CTX_PARAMS_ = 18;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_GETTABLE_CTX_PARAMS_ = 19;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SET_CTX_PARAMS_ = 20;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SETTABLE_CTX_PARAMS_ = 21;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_GET_CTX_MD_PARAMS_ = 22;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_GETTABLE_CTX_MD_PARAMS_ = 23;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SET_CTX_MD_PARAMS_ = 24;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SETTABLE_CTX_MD_PARAMS_ = 25;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_QUERY_KEY_TYPES_ = 26;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SIGN_MESSAGE_INIT_ = 27;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SIGN_MESSAGE_UPDATE_ = 28;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_SIGN_MESSAGE_FINAL_ = 29;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_VERIFY_MESSAGE_INIT_ = 30;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_VERIFY_MESSAGE_UPDATE_ = 31;
openssl_core_dispatch.pas:  OSSL_FUNC_SIGNATURE_VERIFY_MESSAGE_FINAL_ = 32;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_ENCRYPT_INIT_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_ENCRYPT_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_DECRYPT_INIT_ = 4;
```

```
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_DECRYPT_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_FREECTX_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_DUPCTX_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_GET_CTX_PARAMS_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_GETTABLE_CTX_PARAMS_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_SET_CTX_PARAMS_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_ASYM_CIPHER_SETTABLE_CTX_PARAMS_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_ENCAPSULATE_INIT_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_ENCAPSULATE_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_DECAPSULATE_INIT_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_DECAPSULATE_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_FREECTX_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_DUPCTX_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_GET_CTX_PARAMS_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_GETTABLE_CTX_PARAMS_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_SET_CTX_PARAMS_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_SETTABLE_CTX_PARAMS_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_AUTH_ENCAPSULATE_INIT_ = 12;
openssl_core_dispatch.pas:  OSSL_FUNC_KEM_AUTH_DECAPSULATE_INIT_ = 13;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_FREECTX_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_GET_PARAMS_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_GETTABLE_PARAMS_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_SET_CTX_PARAMS_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_SETTABLE_CTX_PARAMS_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_DOES_SELECTION_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_ENCODE_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_IMPORT_OBJECT_ = 20;
openssl_core_dispatch.pas:  OSSL_FUNC_ENCODER_FREE_OBJECT_ = 21;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_NEWCTX_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_FREECTX_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_GET_PARAMS_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_GETTABLE_PARAMS_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_SET_CTX_PARAMS_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_SETTABLE_CTX_PARAMS_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_DOES_SELECTION_ = 10;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_DECODE_ = 11;
openssl_core_dispatch.pas:  OSSL_FUNC_DECODER_EXPORT_OBJECT_ = 20;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_OPEN_ = 1;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_ATTACH_ = 2;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_SETTABLE_CTX_PARAMS_ = 3;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_SET_CTX_PARAMS_ = 4;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_LOAD_ = 5;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_EOF_ = 6;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_CLOSE_ = 7;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_EXPORT_OBJECT_ = 8;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_DELETE_ = 9;
openssl_core_dispatch.pas:  OSSL_FUNC_STORE_OPEN_EX_ = 10;
openssl_crypto.pas:  OPENSSL_VERSION_ = 0;
openssl_idea.pas:  IDEA_ENCRYPT_ = 1;
openssl_pkcs7.pas:  PKCS7_STREAM_ = $1000;
openssl_rc2.pas:  RC2_ENCRYPT_ = 1;
openssl_rc2.pas:  RC2_DECRYPT_ = 0;
openssl_ssl.pas:  SSL_TXT_kECDHE_ = 'kECDHE';
openssl_ssl.pas:  SSL_TXT_ADH_ = 'ADH';
openssl_ssl.pas:  SSL_TXT_AECDH_ = 'AECDH';
openssl_store.pas:  OSSL_STORE_SEARCH_BY_NAME_ = 1;
openssl_store.pas:  OSSL_STORE_SEARCH_BY_ISSUER_SERIAL_ = 2;
openssl_store.pas:  OSSL_STORE_SEARCH_BY_KEY_FINGERPRINT_ = 3;
openssl_store.pas:  OSSL_STORE_SEARCH_BY_ALIAS_ = 4;
```

Appendix B - Allow Nil Functions

Only a small number of functions are currently allowed to be nil on load, and are limited to legacy support functions for TLS and SSL methods in OpenSSL 3.x. These are:

TLSv1_method
TLSv1_server_method
TLSv1_client_method
TLSv1_1_method
TLSv1_1_server_method
TLSv1_1_client_method
TLSv1_2_method
TLSv1_2_server_method
TLSv1_2_client_method
SSLv3_method
SSLv3_server_method
SSLv3_client_method
SSLv23_method
SSLv23_server_method
SSLv23_client_method